

of the SPI slave is used to find an appropriate chip driver, and then the probe function of the chip driver is called.

The chip driver probe function receives *platform_data* information registered by the *spi_register_board_info()* function. This information is used by the chip driver to define partitions of the flash that are available through */proc/mtd*.

You can arrive at the skeleton of a typical SPI chip driver as follows:

```
int m25p_probe(struct spi_device *spi)
{
    struct flash_platform_data *data;
    data = spi->dev.platform_data;

    struct flash_info *info;
    /* find the actual chip from list of chips
    stored in m25p_ids[]
    * A sample entry in m25p_ids[] looks like
    * { "m25pe80", INFO(0x208014, 0, 64 * 1024,
    16, 0) },
    * The info pointer is assigned to such an
    entry of matching chip
    */

    /* create and initialize MTD device */
    struct mtd_info *mtd = kzalloc(sizeof(struct mtd_
info), GFP_KERNEL);
    mtd->name = data->name;
    mtd->type = MTD_NORFLASH;
    mtd->writesize = 1;
    mtd->flags = MTD_CAP_NORFLASH;
    mtd->size = info->sector_size *
info->n_sectors;
    mtd->erase = m25p80_erase;
    mtd->read = m25p80_read;
    mtd->write = m25p80_write;
    mtd->erasesize = info->sector_size;
    mtd->dev.parent = &spi->dev;

    /* register MTD device */
    int parts = data->parts;
    int nr_parts = data->nr_parts;
    return add_mtd_partitions(mtd,
parts, nr_parts);
}
```

At this stage, the SPI slave device is ready to be accessed by *MTD utils*. Also the */proc/mtd* file provides some useful information about the MTD device.

The SPI controller driver: Operation

In this section, let us examine the data transfer to the SPI slave device. There are two important things to understand here:

- Communication between the chip driver and the SPI controller driver
- The transfer method of the SPI controller driver

Communication between the chip driver and SPI controller driver: The communication between the chip driver and SPI controller driver takes place using two important structures: (i) *struct spi_message*; (ii) *struct spi_transfer*.

The *spi_message* represents a single SPI message between the chip driver and controller driver. The SPI message may contain one or two SPI transfers. The SPI transfer represents one operation. For example, when the chip driver wants to read from the SPI flash, it has to create one SPI transfer for the 'write command' and a second SPI transfer for the 'buffer to read data in'. These two transfers are embedded in one SPI message, and this SPI message is sent to the SPI framework. The SPI framework delivers this SPI message to the transfer method of the SPI master.

Therefore, the SPI transfers in SPI messages can be referred to as 'CMD Transfers' and 'RSP Transfers'. Each SPI transfer contains two buffers *tx_buf* and *rx_buf*, which may be NULL if not relevant to the command. Table 1 shows a typical mapping.

As shown above, some SPI commands send only one SPI transfer (CMD Transfer) in the SPI message, while some send two SPI transfers (CMD Transfer and RSP Transfer) in the SPI message. The SPI transfer method of the SPI controller should be ready to deal with this.

Let's look at an example of how the chip driver creates SPI messages and SPI transfers, and sends them to the SPI controller.

Table 1: SPI command to SPI Transfer's buffer mapping

SPI command	CMD Transfer		RSP Transfer	
	tx_buf	rx_buf	tx_buf	rx_buf
Write Status Register	Buffer containing opcode	NULL	NULL	Buffer to write to status register
Page Program	Buffer containing opcode	NULL	Data buffer to be written to flash	NULL
Read Data Bytes	Buffer containing opcode	NULL	NULL	Buffer to read data bytes
Read Status Register	Buffer containing opcode	NULL	NULL	Buffer to read status register
Erase Whole Flash Chip	Buffer containing opcode	NULL	N/A	
Sector Erase	Buffer containing opcode	NULL	N/A	
Read JEDEC ID	Buffer containing opcode	NULL	NULL	Buffer to read JEDEC ID
Write Enable	Buffer containing opcode	NULL	N/A	